

Aula 08 — Processos (parte 3): Servidores e Migração de Código

Resumo 9 de 14 · arquivo aula08 · 29 imagens / 21 slides · Cap. 3 de van Steen & Tanenbaum

Fecha o tema Processos com dois assuntos: como **servidores** são organizados (iterativo × concorrente, com/sem estado, servidores de objetos, clusters e CDNs) e por que/como se faz **migração de código** (mover a computação de uma máquina para outra).

1. Servidores: organização geral

Um **servidor** é um processo que implementa um serviço: espera uma requisição, garante que seja atendida e volta a esperar. Dois tipos:

- **Iterativo:** trata a requisição inteira antes de pegar a próxima.
- **Concorrente:** um *dispatcher* repassa cada requisição para uma thread/processo separado. É a norma, pois lida bem com várias requisições e com operações bloqueantes (disco, outros servidores).

Contatar um servidor: a maioria dos serviços está ligada a uma **porta** conhecida (ftp 20/21, telnet 23, smtp 25, HTTP 80); portas também podem ser atribuídas dinamicamente. **Comunicação out-of-band** resolve como interromper um servidor que já aceitou uma requisição: (1) uma porta separada para mensagens urgentes (exige escalonamento por prioridade) ou (2) as facilidades do transporte (ex.: mensagens urgentes do TCP na mesma conexão, capturadas por sinalização do SO).

Servidores com e sem estado

Tipo	Características
Sem estado (stateless)	Não guarda estado do cliente após atender. Cliente e servidor ficam independentes ; menos inconsistência em caso de falhas; mas pode perder desempenho (não antecipa o comportamento, ex.: pré-carregar blocos). Meio-termo: <i>soft state</i> .
Com estado (stateful)	Conhece o estado/histórico do cliente: permite pré-carregamento e que o cliente mantenha cópias locais em cache . Desempenho pode ser altíssimo; confiabilidade geralmente não é problema.

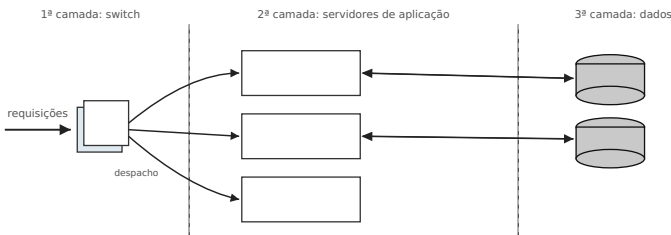
Perguntas para pensar (ficam na aula): comunicação orientada a conexão combina com design sem estado? E cookies — são *stateful* ou *stateless*?

Servidores de objetos e Apache

Um **servidor de objetos** hospeda objetos e decide, via uma **política de ativação**, o que fazer quando chega uma invocação: onde estão código e dados, se o objeto é transiente (existe só enquanto o servidor existir), como usar threads, se mantém o estado modificado. Quem implementa essa política é o **adaptador de objeto** (exemplo na aula: o sistema Ice). O **Apache** se apoia no APR (Apache Portable Runtime) para abstrair o SO.

Clusters de servidores

Organização comum em **três camadas**: um switch lógico despacha as requisições, servidores de aplicação/computação as processam, e um sistema de arquivos/BD distribuído guarda os dados.



Cluster em 3 camadas: o switch da 1ª camada faz o **despacho de requisições** para os servidores de aplicação, que acessam o sistema de dados distribuído.

Fazer a 1ª camada tratar *toda* a comunicação vira gargalo — uma solução é o **TCP handoff** (o switch passa a conexão para o servidor escolhido, que responde direto ao cliente). Quando os servidores estão **espalhados pela Internet**, o despacho costuma usar **DNS**: o cliente consulta o serviço, e o DNS (sabendo das réplicas e do IP do cliente) devolve o servidor mais próximo — problema: o resolvedor DNS pode estar longe do cliente real. É a base das **CDNs** como a Akamai, cujo cache é sofisticado a ponto de guardar não só dados, mas também parte do código da aplicação.

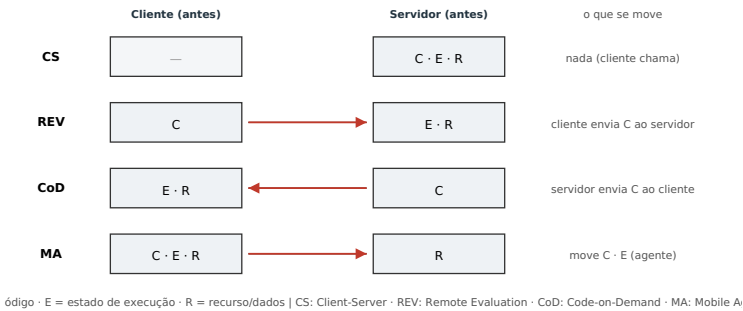
2. Migração de código

Motivos para mover código de uma máquina para outra:

- **Distribuição de carga:** equilibrar o uso do datacenter (evitar desperdício de energia) e aproximar a computação de onde estão os dados.
- **Flexibilidade:** mover código ao cliente quando necessário, evitando pré-instalação e permitindo configuração dinâmica.
- **Privacidade/segurança:** quando os dados *não podem* sair do lugar (motivos legais), move-se o **código para os dados** — como no aprendizado de máquina federado.

Paradigmas de mobilidade de código

Um processo tem três partes: **código** (C), **estado de execução** (E) e **recursos/dados** (R). Onde cada parte começa e o que se move define quatro paradigmas:



Quatro paradigmas: no CS nada se move (RPC clássico); no REV o cliente manda o código ao servidor; no CoD o servidor manda o código ao cliente (ex.: applet/JS); no MA a unidade em execução migra (agente móvel).

Mobilidade fraca x forte

Um processo tem **segmento de código**, **segmento de dados** e **estado de execução** (contexto da thread).

- **Mobilidade fraca:** move só código + dados e **reinicia** a execução. Simples (ainda mais se o código for portátil). Distingue-se *push* (enviar código) de *pull* (buscar código).
- **Mobilidade forte:** move tudo, **inclusive o estado de execução**. Duas variantes: *migração* (move o processo inteiro para outra máquina) e *clonagem* (inicia um clone no mesmo estado).

Migração em sistemas heterogêneos e de VMs

O problema em ambientes heterogêneos: a máquina de destino pode não conseguir rodar o código, pois o contexto depende do hardware/SO/runtime. A saída é uma **máquina abstrata** implementada em várias plataformas — linguagens interpretadas (que têm sua própria VM) ou VMMs. Containers, por dependerem diretamente do SO, são difíceis de migrar entre plataformas diferentes.

Ao **migrar uma VM**, há três estratégias para a memória: (1) enviar as páginas e reenviar as que forem modificadas durante a migração; (2) parar a VM, migrar a memória e reiniciar; (3) iniciar a nova VM já e deixá-la **puxar as páginas sob demanda**. Cuidado com o desempenho: uma migração completa pode levar **dezenas de segundos**, e o serviço fica indisponível por vários segundos durante o processo.